

ML Specification Report — playground-series-s3e7

Hotel Reservation Cancellation Prediction · our from-scratch agent (GBDT pool + tree-search v3)

Figures grounded in the competition's `facts.json`, `eda_summary.json`, `STATUS.md`, and `experiments_tree_v3.json`, with the head-to-head benchmark drawn from `ext_facts.json`.

Overview

The task is to rank 28,068 hotel reservations by cancellation probability (metric: ROC-AUC). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (LightGBM / XGBoost / CatBoost) refined first by a linear self-improvement loop and then by a 60-node tree search (`harness_v3`) that ends on a 38-member kitchen-sink mega-blend. Our tree-search champion (node #48) scores **CV OOF AUC 0.900455**; the submission-ready linear-iteration final sits at ≈ 0.8999 , and that is the figure the NVIDIA reproduce-agent is measured against — NVIDIA copies a public EDA-and-submission kernel (verbatim) to reach 0.9014, a **marginal +0.17%** gap that `ext_facts.json` scores as a **tie**.

Why it matters. Reservation cancellations drive revenue loss and overbooking risk in hospitality; predicting them enables dynamic pricing, overbooking control, and targeted retention.

Data

Purpose of Data. Predict whether a hotel reservation is cancelled (`booking_status` = 1) — a binary-classification Playground Series episode (Season 3, Episode 7). **Data Format** is clean **tabular CSV**; every one of the 17 raw columns arrives already integer/float-encoded (meal plan, room type, and market segment are pre-label-encoded, so there are no string categoricals to parse). **Data Volume** is **42,100 training rows / 28,068 test rows**, with 17 raw predictive features once `id` and the target are removed.

Data Quality is high: **no missing values** in either split (train and test missing maps are both empty), and no meaningful train ↔ test distribution shift (`STATUS.md` reports all normalized shifts < 0.02, so the largest percentage moves in `eda_summary` sit on near-constant rare columns). One quality figure is **in conflict across sources and we report it honestly rather than pick one**: `eda_summary.json` records **562 duplicate training rows** (`duplicate_rows_train` = 562), whereas `STATUS.md` states there are "**no duplicate rows/ids**". The two were not reconciled in this run; the modelling was performed on the full 42,100-row train set regardless. The one modelling wrinkle is a mild class imbalance — **39.2% of reservations were cancelled** (`booking_status` mean 0.392019; value counts 0 = 25,596 / 1 = 16,504) — which motivates the metric (AUC) and the StratifiedKFold choice. The strongest EDA signals, all consistent with domain intuition, were:

- `lead_time` is the single dominant predictor (single-feature AUC 0.73, Pearson +0.375 / Spearman +0.390); the cancellation rate rises monotonically from 11% (lead ≤ 7 days) to 68% (lead > 180 days).

- **Protective signals:** `no_of_special_requests` (Pearson -0.220), `repeated_guest` (Pearson -0.136 — repeated guests cancel only 0.9% of the time vs ~40% for new guests), and `required_car_parking_space` (Pearson -0.093).
- `market_segment_type` carries a large rate spread (Spearman $+0.187$): segment 1 (likely "Online") cancels 50.5% of the time vs 1.6% for segment 4.
- `avg_price_per_room` is non-monotonic (rises then dips in the top price quintile; Pearson $+0.157$) and was kept raw, not binned; `arrival_date` / `arrival_month` carry near-zero raw correlation (0.003 / 0.008) and proved noise-like.
- No leakage: every single-feature AUC is < 0.85 (max was `lead_time` at 0.73, a legitimately strong non-leaky signal).

Annotation Guidelines. The label is `booking_status` $\in \{0, 1\}$ (1 = cancelled). Submissions are **cancellation probabilities** scored by AUC — a ranking, not a hard 0/1 decision.

Feature Set. From the 17 raw columns we iterated two engineered variants; the champion pipeline uses the **iter-2 trimmed 25-feature set** (17 raw + 8 engineered) after an earlier 31-feature variant regressed below baseline:

Group	Features
Raw (17)	booking counts, nights, meal/room/segment codes, <code>lead_time</code> , arrival year/month/date, guest history, <code>avg_price_per_room</code> , special requests
Engineered — kept (8)	<code>total_nights</code> , <code>total_guests</code> , <code>has_children</code> , <code>weekend_ratio</code> , <code>price_per_person</code> , <code>lead_time_log</code> , <code>prior_cancel_rate</code> , <code>has_prior_history</code>
Engineered — dropped in iter-2 (6)	<code>price_per_night</code> , <code>month_sin</code> , <code>month_cos</code> , <code>date_sin</code> , <code>date_cos</code> , <code>no_special_and_price</code> (cyclical encodings + a weak interaction, all noise-like)

Splitting strategy. A single canonical **Stratified 5-Fold CV (seed = 42)**, shuffled, stratified on `booking_status` directly (binary target — no binning needed), shared across every tier so scores are directly comparable. Fold AUCs were stable across all three base models (0.893–0.901 range). The leaderboard was **not exercised this run** (`leaderboard: null` in `facts.json`; no Kaggle credentials available), so all scores here are **CV-only**.

Models & Architecture

Purpose of Architecture. A binary cancellation classifier that maximizes ROC-AUC. **Architecture Type** is a **gradient-boosted decision-tree ensemble** — LightGBM as the primary learner, with XGBoost and CatBoost members — combined by a weighted blend; the champion is a **38-member mega-blend (9 nonzero-weight survivors)** discovered at tree-search v3 **node #48**.

Each member takes the same **Input Format** — a numeric feature matrix with the pre-encoded categoricals left as integer codes — of **Input Dimension 25 features** per row (the iter-2 trimmed set; a 1-D vector, no spatial/sequence structure).

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by histogram-based gradient boosting (representative Optuna-tuned LGB: `num_leaves` = 178, `max_depth` = 3, `learning_rate` = 0.068, `reg_alpha` = 2.14, up to 3000 boosting rounds with early stopping; the hand-set diversity LGB used `num_leaves` = 63, `lr` = 0.03). The blend layer sits on top as a convex weighted average of the members' out-of-fold predictions, with weights found in the champion by a rank-space **Dirichlet(k = 800) + coordinate-ascent** search over the full 38-member pool. **Model Complexity** is best expressed as trees × leaves rather than a dense parameter count: a single LGB member is on the order of thousands of leaves, and the champion blends a 38-member pool of which the weight search kept 9. The surviving weights: **XGBDIV family 0.459** (4 diversity-XGB nodes), **SEEDBAG 0.358** (seed-bagged tuned LGB), **EXPL_BOUND2 0.114** (a boundary-push depth-2 LGB — solo only 0.897541 yet the 3rd-largest weight), **root 0.038**, **FEATPRUNE-child 0.018**, **XGBHAND-child 0.013** — confirming that blend contribution ≠ solo score.

Training procedures

Training proceeds as a **linear self-improvement loop followed by a tree search**, all on the shared 5-fold split and measured on the same OOF scale:

Tier	Configuration	OOF AUC
Generic baseline (exp 1)	3-model blend, raw 17 features	0.89882
Iter-1 (exp 2)	17 raw + 14 engineered (31 feats)	0.89788
Iter-2 / reflexion (exp 3)	trimmed to 25 feats (8 engineered)	0.899395
Phase B R1 (exp 4)	+ Optuna-tuned LGB (50-trial, fold-0 proxy)	0.899891
Phase B R2 (exp 5, REJECTED)	+ Optuna-tuned XGB (diversity loss)	0.899722
Phase B R3 final (exp 6)	prob 0.01-grid blend refine	0.899893
Tree-search v2 sweep (D-3)	<code>harness_v2</code> , 22 evals	0.900242
Tree-search v3 (F-2), node #48	<code>harness_v3</code> , 60 evals, best at eval 46	0.900455

The **Loss Function** is binary log-loss (LightGBM `binary` objective; XGBoost `binary:logistic`; CatBoost `Logloss`) with model selection on AUC. Phase B R2 is an instructive reject: retuning XGB improved its solo (0.898765 → 0.898860) but *regressed* the blend (−0.000169) by collapsing ensemble diversity — so the hand-set XGB was kept. The **Optimization Algorithm** is gradient boosting (histogram-based) — **not** SGD/ADAM — with blend weights optimized by grid search (linear tiers) and Dirichlet + coordinate-ascent (tree search). The **Learning Rate** is the boosting shrinkage (Optuna-tuned LGB `learning_rate` = 0.068; hand-set members = 0.03); there is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by CV early-stopping, not a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched).

Training Duration for the tier that produced the champion was **1,593 s of wall-clock across 60 tree-search nodes** (`our_wall_s`; `our_train_fits` = 60), with the best node found at evaluation 46; the Optuna LGB tune was 50 trials / 300.2 s and a solo LGB fits in ~24–30 s. **Training Memory** was **not recorded** — a 42k

× 25 float matrix is trivial on the single arm64 CPU machine and peak was not separately measured. **Transfer Learning** is *N/A (no pretrained weights)*; its analogue here is **cross-competition experience-library priors** injected into the tree search (P0–P19 bullets), which materially helped ensemble mechanics (informed-mutation win rate 62.5% vs 16.7% in the v2 sweep) though comp-specific feature pruning did **not** transfer. **Data Augmentation** is *N/A (tabular)*; the analogues are feature engineering (the 8 kept features) and seed-bagging (LGB seeds 42 → 2024).

Reproducibility Standards are strict: fixed seeds (fold seed = 42; seed-bag 2024 / 4001; Optuna seeds logged), and digit-for-digit reproduction of every known config was verified before the search ran (root LGB_tuned 0.899215, XGB_hand 0.898765, CAT_hand 0.896709, all exact to 6 dp). The champion is a **native v3 run — no replay was needed** (`faithful_replay`: "native v3 run (no replay needed)"; `v3_native` = true). One honest caveat carried from `STATUS.md`: blend weights are fit on the full OOF with no nested validation, so the ~0.0002-level gain over the v2 sweep carries OOF-weight-overfit risk — the qualitative direction (burst mega-blend > hand-grown blend) is the robust part, the 4th decimal is not.

Inference procedures

Decision Threshold is *N/A* — AUC is a ranking metric, so we submit raw probabilities and never threshold (a 0.5 cut would only be needed if hard labels were required). **Inference Duration** was **not recorded**; GBDT scoring of the 28,068 test rows is sub-second per member on CPU. **Inference Memory** was **not recorded** and is negligible (tree inference is lightweight, not capped). Note that the tree-search champion (node #48) was an **OOF-only search result** — no test predictions were generated for it, so no champion submission file exists; the submission-ready artifact is the linear-iteration final (exp 6, `sub_blend_0.89989_20260703_205132.csv`).

Evaluation & Benchmarking

Task for Performance Evaluation. Rank the test reservations by cancellation probability. **Performance Metrics.** ROC-AUC is the sole competition metric; our tree-search champion reaches **CV OOF AUC 0.900455**, climbing 0.89882 → 0.899395 → 0.899893 (linear final) → 0.900242 (v2) → 0.900455 (v3) across the trajectory. Per `ext_facts.json`, the **submission-ready our_best used for benchmarking is ≈0.8999** (the linear-iteration final, exp 6). This run posted **no Kaggle leaderboard score** (local CV only; `leaderboard`: null), so there is no CV ↔ LB gap to report.

Performance Benchmarking. The natural comparator is the **NVIDIA reproduce-agent**, which copies the strongest public kernel verbatim:

Agent	Approach	AUC	Note
NVIDIA	reproduces <code>sergiosaharovskiy/ps-s3e7-2023-eda-and-submission</code> (verbatim)	0.9014	marginal +0.17%
Our agent	from-scratch GBDT pool + tree-search v3	≈0.8999	tie

NVIDIA is numerically ahead by a **marginal +0.17%** (0.9014 vs ≈0.8999), but `ext_facts.json` scores the outcome a **tie** — the gap sits within noise for this AUC-flat, near-saturated hotel-cancellation problem,

where the linear loop squeezed only $\sim +0.0005$ total and even the 60-node tree search added ~ 0.0006 more. As on the newer seasons, the reproduce-vs-originate contest here is effectively a dead heat: the well-optimized public kernel and our from-scratch pipeline land on the same fourth decimal.